Mycroft.ai

A configurable Responsible AI control plane.

Team Mycroft.ai · IIT Bombay · Accenture Innovation Challenge 2026, Round 2

Submitted against **Track 1: ControlPlane.ai**

A note on names. **Mycroft.ai** is the product. **ControlPlane.ai** is Accenture's name for the problem track, and appears here only when referring to the brief. The `controlplane/` package is named for the architecture — a control plane sitting between the enterprise's AI systems and its users — which is the term the solution itself uses.

> **Nothing in this repository is real enterprise data.** Every trace is simulated, every identifier is fabricated (with valid checksums, so the deterministic detectors are exercised realistically, but corresponding to no real person), and every rupee figure is computed from assumptions that are published in `docs/ASSUMPTIONS.md` and printed next to the figure itself.

Run it

```
python3 -m venv .venv && ./.venv/bin/pip install -r requirements.txt
bash run.sh
```

`run.sh` generates the corpus, produces the evidence pack, closes one defect loop end to end, and opens the prototype at `http://localhost:8501`. **No API key and no network are required.**

Individually:

```
python data/generate.py      # 264 simulated traces, seeded
python -m data.holdout       # 30 hand-written adversarial traces
python scripts/run_eval.py   # precision/recall/FP/FN, latency, defect pricing
python scripts/run_demo.py   # the seven-step lifecycle, headless
streamlit run app/streamlit_app.py
```

What it does

Enterprises run several AI systems at once — a customer-facing support assistant, an internal copilot, a decision-support tool in a regulated workflow — and each has a different risk signature, latency budget and regulatory obligation. Mycroft.ai checks every interaction, routes deeper evaluation where the risk justifies the cost, and turns what it finds into a **closable defect**: priced, evidence-pinned, resolved by a human, carried into a versioned contract, and proved by a regression run that also reports what the fix cost.

```
Jurisdiction → Policy Pack → Evaluation Contract → Detector configuration

interaction ─▸ FAST LANE   deterministic · inline · 100% of traffic
                           PII (Verhoeff/Luhn validated) · deny lists · loop breakers
            ─▸ SLOW LANE   semantic · asynchronous · never blocks the user
                           grounding · counterfactual bias screen · explanation policy
            ─▸ FUSION      every label recorded, one strictest action taken
            ─▸ DEFECT      clustered by failure signature, priced, evidence-pinned
            ─▸ HUMAN       a reviewer confirms or rejects; nothing moves without this
            ─▸ CONTRACT    versioned, content-hashed, git-diffable
            ─▸ REGRESSION  re-run the defect's traces; report benefit *and* cost
            ─▸ AUDIT       hash-chained; a record cannot be edited afterwards
```

Three principles the code actually enforces:

1. **We never silently rewrite model output.** Detect, explain with pinned evidence, then allow / log / route to a human / block.
2. **The evaluator is never the generator marking its own homework.** The optional Anthropic judge is a different model family; the default judge is not a language model at all, and never calls itself one.
3. **Every number is measured or labelled.** Including the ones that make us look worse.

Measured results

From `outputs/metrics.md`, reproducible from a clean clone.

Generated corpus, 264 traces, jurisdiction IN:

RISK FAMILY	PRECISION	RECALL	FP RATE	FN RATE
Privacy	0.957	1.000	0.9%	0.0%
Hallucination	0.965	0.887	1.0%	11.3%
Bias	1.000	1.000	0.0%	0.0%
Policy	0.800	1.000	2.5%	0.0%
Behaviour	1.000	1.000	0.0%	0.0%

Hand-written adversarial holdout, 30 traces, written *after* the detectors were built: privacy 1.00 / 1.00, policy 1.00 / 1.00, bias 1.00 / 1.00, and **hallucination 0.857 precision at 0.750 recall**. That gap is one specific class — open-ended fabrication with no figure to contradict — and it is the clearest case in the system for routing to an LLM judge.

Latency: Fast Lane p95 is **under 0.05 ms** of detector compute against a 100 ms contract budget (it varies slightly per run, since it is wall-clock timing). Compute only; excludes network and serialisation.

The same 264 traces under three policy packs, no code changes: 89 blocks under IN and US, **112 under EU**.

One closed loop: failures reaching the user **6 → 3 (−50%)** on a 30-trace regression set, at a cost of **+11 false positives** and **+4 human reviews** across 114 interactions.

Layout

```
contracts/           Evaluation Contracts and jurisdiction policy packs (YAML)
  use_cases/         customer_support, internal_knowledge, decision_support
  policy_packs/      IN (DPDP + RBI FREE-AI), EU (GDPR + AI Act), US (sectoral)
  versions/          frozen contract versions written by the fix loop; diff in git
controlplane/        the control plane
  contract.py        policy-pack merge (a ratchet), content-hashed versions
  trace.py           Trace / Finding / Decision schema
  corpus.py          the governed documents a response is checked against
  fastlane/          pii.py · lists.py · loops.py   (deterministic, inline)
  slowlane/          grounding.py · bias.py · policy.py · judge.py  (semantic, async)
  risk.py            two-lane pipeline, overlap fusion, action resolution
  defects.py         clustering, root-cause hypotheses, pricing
  lifecycle.py       resolve → contract patch → regression → cost
  audit.py           hash-chained append-only trail
  metrics.py         precision/recall/FP/FN, latency, threshold sweeps
data/                corpus documents, seeded generator, hand-written holdout
app/                 the Streamlit prototype
scripts/             run_eval.py (evidence pack) · run_demo.py (the loop)
outputs/             everything generated: metrics, latency, defects, audit
docs/                proposal · deck · assumptions · methodology · demo · checklist
```

Documents

Business proposal	Problem, solution, users, business case, roadmap, risks
Pitch deck content	15 slides, speaker-ready
Assumptions register	Every number: measured, simulated or assumed — and the Round 1 figures we withdrew
Methodology	How each detector works and where it fails
Demo script	Live run-of-show, video plan, and the hard questions
Brief → evidence map	Every complexity in the brief, and where we address it
Video brief	Setup and click-by-click recording instructions, for someone with no context
Full demo script	The narration word for word, timed to the prototype

Ready-to-submit files live in `dist/` — the proposal as a PDF and as a 17-slide deck, plus PDF renderings of the README, methodology and assumptions register. All of them regenerate from source:

```
python scripts/build_pdf.py
node scripts/build_deck.js dist/Mycroft_AI_Business_Proposal_Deck.pptx
```

Optional: route the Slow Lane to Claude

```
export ANTHROPIC_API_KEY=...
export MYCROFT_JUDGE=anthropic
python scripts/run_eval.py
```

The judge becomes an LLM from a different model family than the generator. Reported metrics in `outputs/` come from the offline path so that they stay reproducible for anyone without a key. If the API call fails, the judge falls back to the offline path — it fails safe, never open.